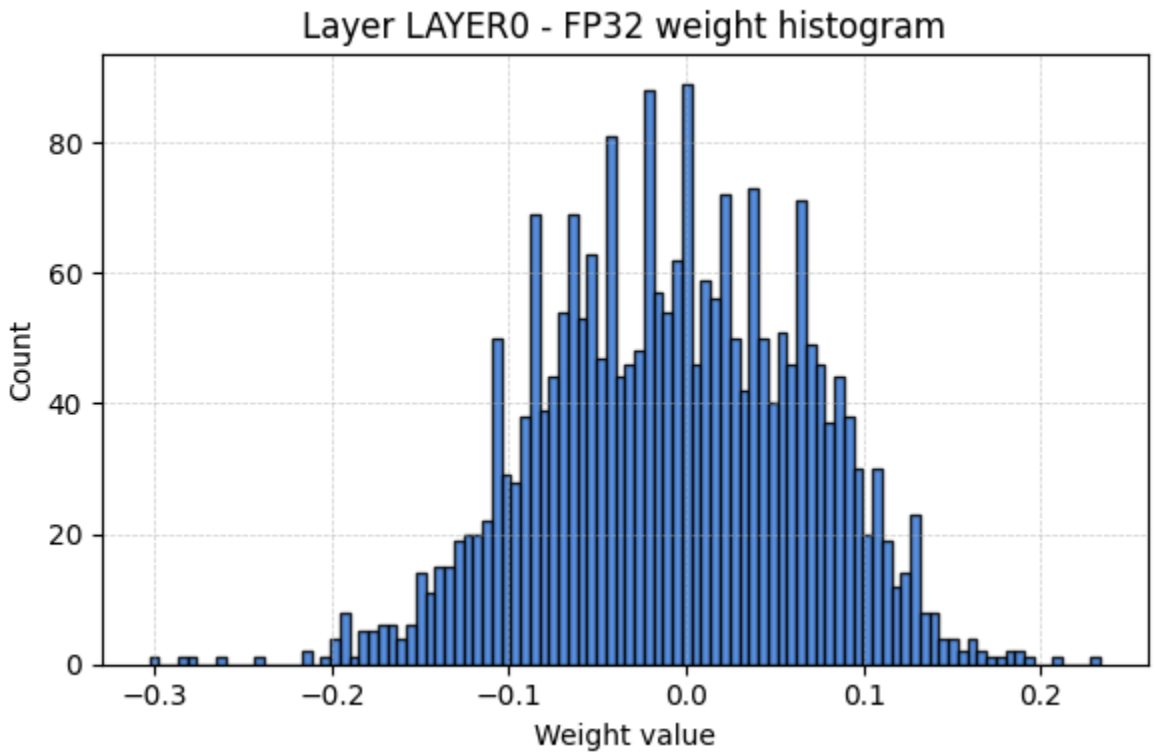
# CPRE 4870/5870 - Lab 04

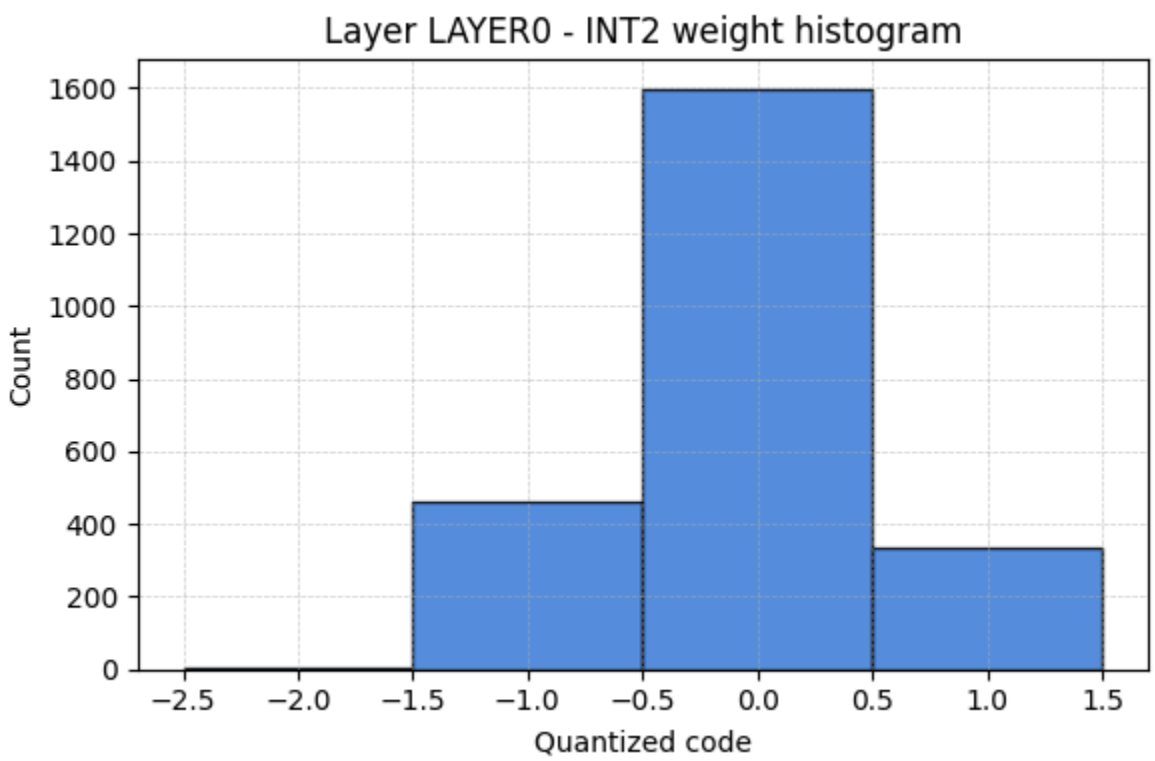
Cameron Gilbertson, Tyler Bibus

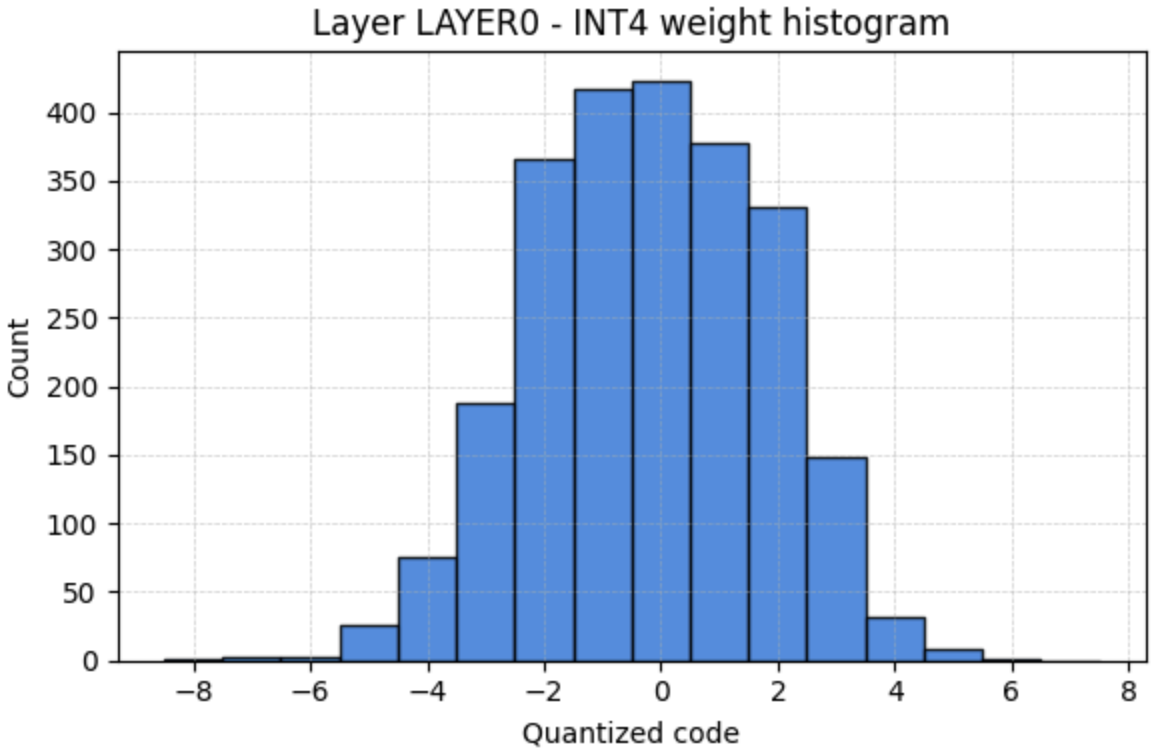
## Overview MAC Comparison Table

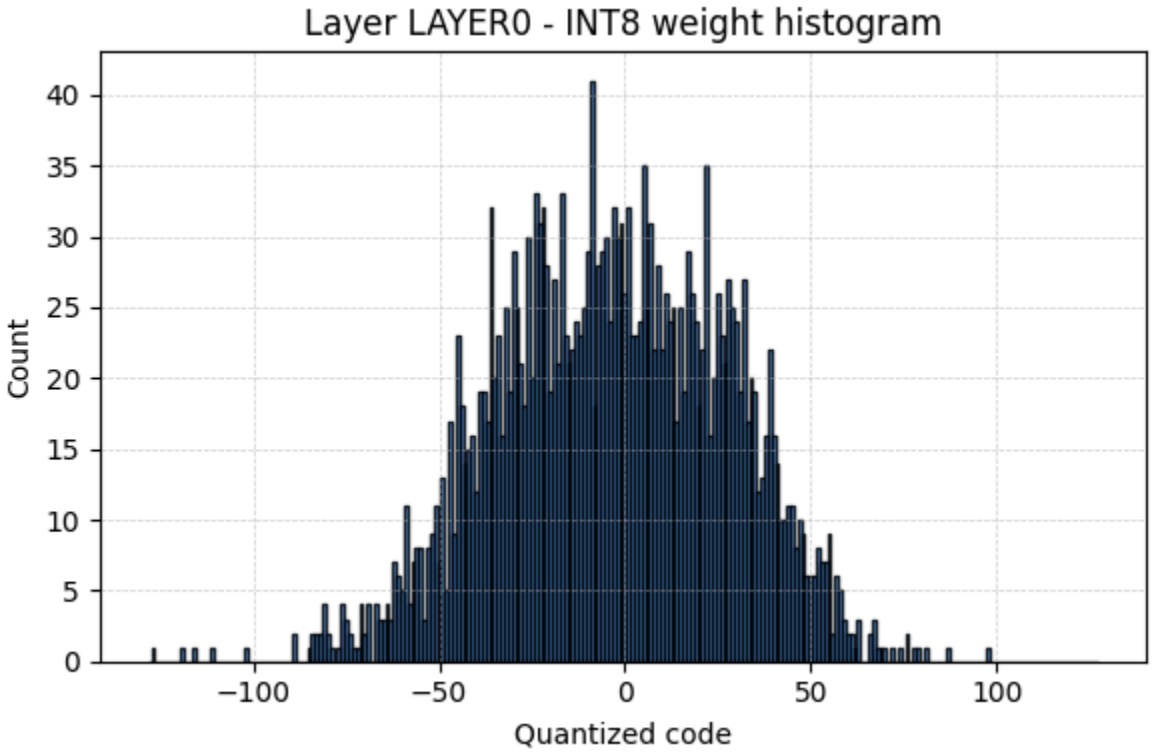
| | **8 Bit Staged** | **8 bit Piped** | **4 bit Staged** | **4 bit Piped** | **2 bit Staged** | **2 bit Piped** | **Variable MAC** |
|---|---|---|---|---|---|---|---|
| **Power** | 1.543 W | 1.543 W | 1.542 W | 1.543 W | 1.543 W | 1.542 W | 1.544 W |
| **LUT** | 1267 | 1260 | 1225 | 1219 | 1204 | 1201 | 1297 |
| **DSP** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **WNS** | 0.627 ns | 2.162 ns | 2.037 ns | 2.550 ns | 2.369 ns | 2.250 ns | 0.688 ns |

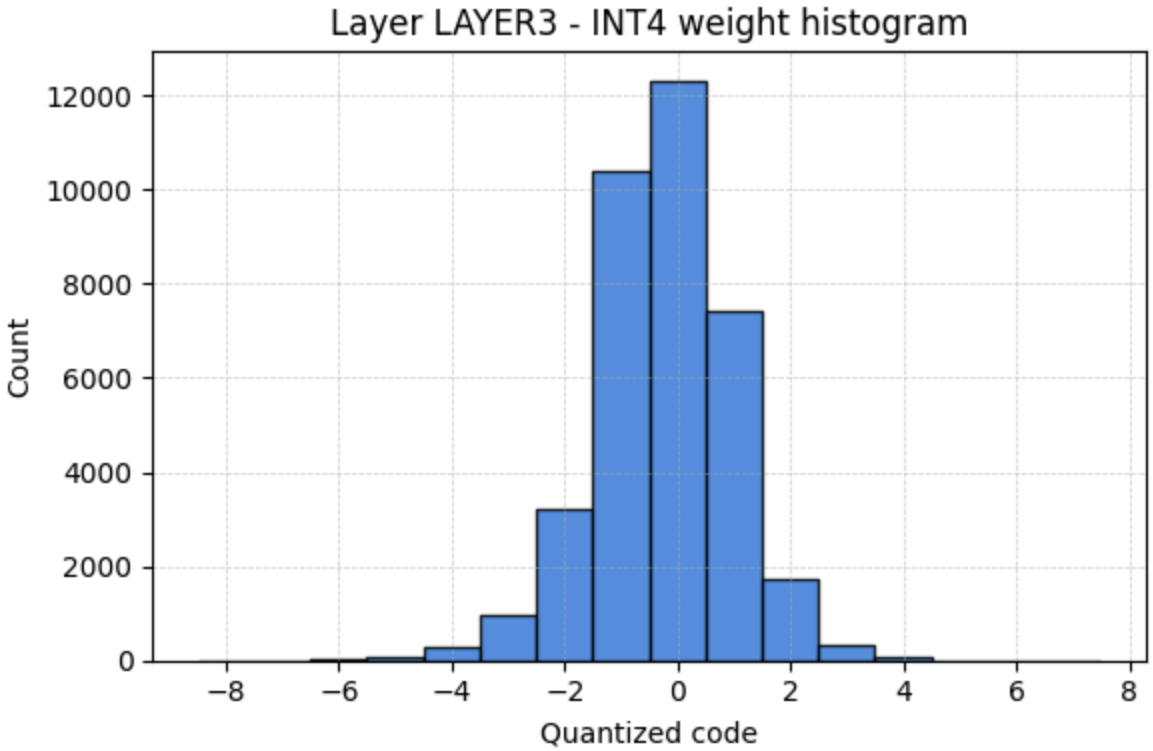
## Weight Histograms

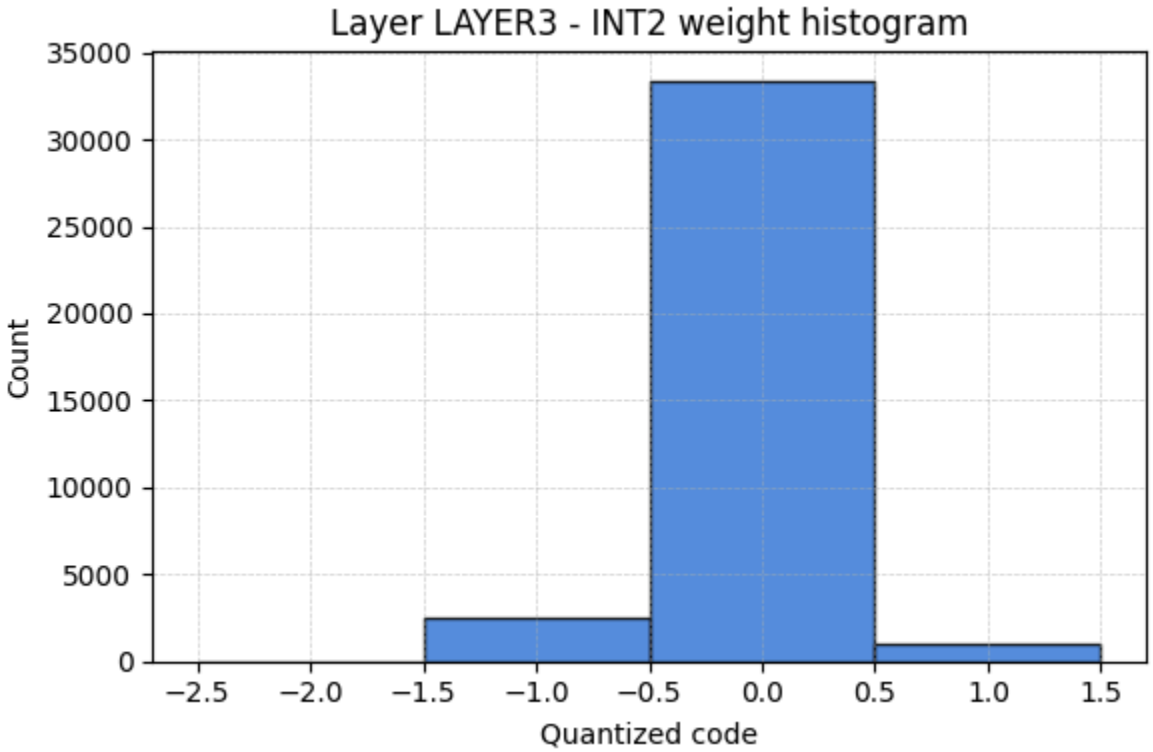


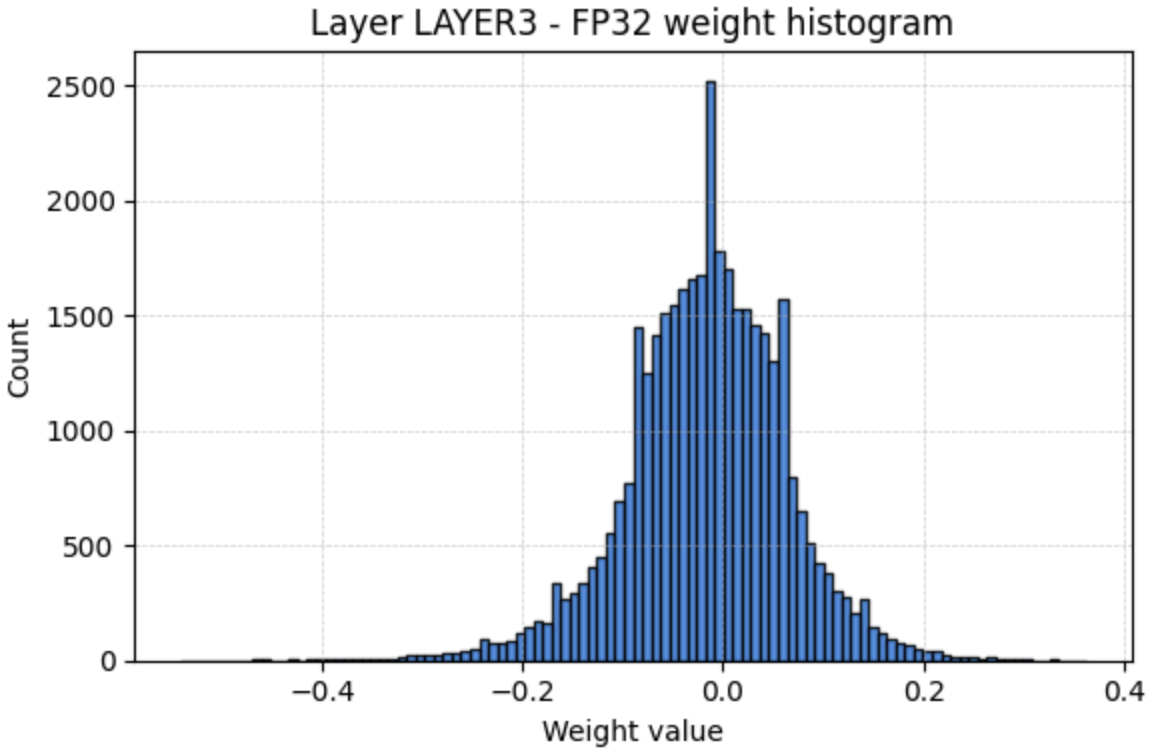



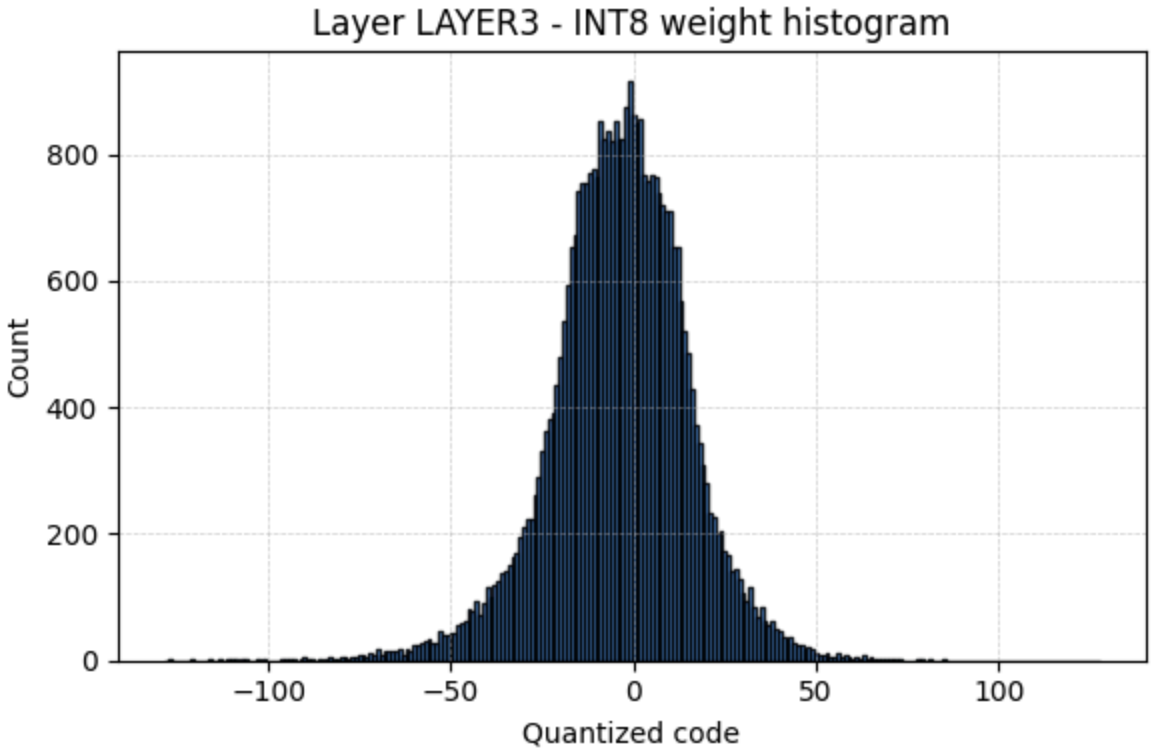

Layer LAYER0 - INT4 weight histogram
Count
Quantized code
0
50
100
150
200
250
300
350
400
−8
−6
−4
−2
0
2
4
6
8

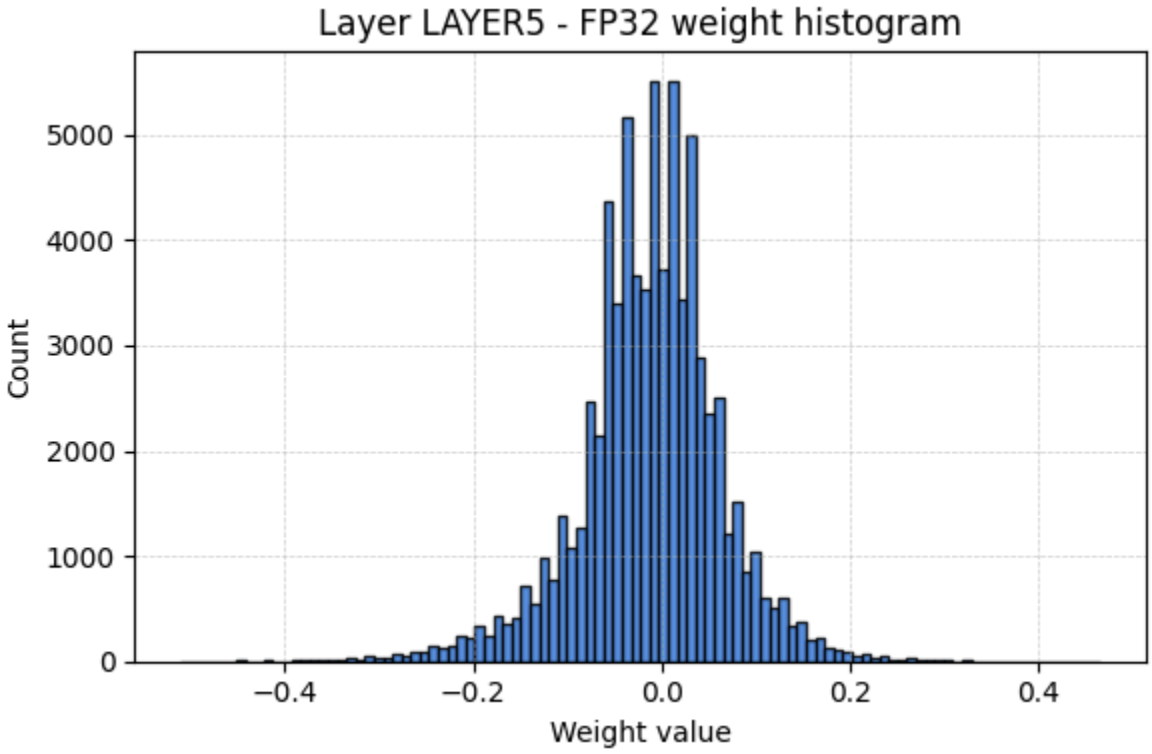

Layer LAYER0 - INT8 weight histogram
Count
Quantized code
0
5
10
15
20
25
30
35
40
−100
−50
0
50
100

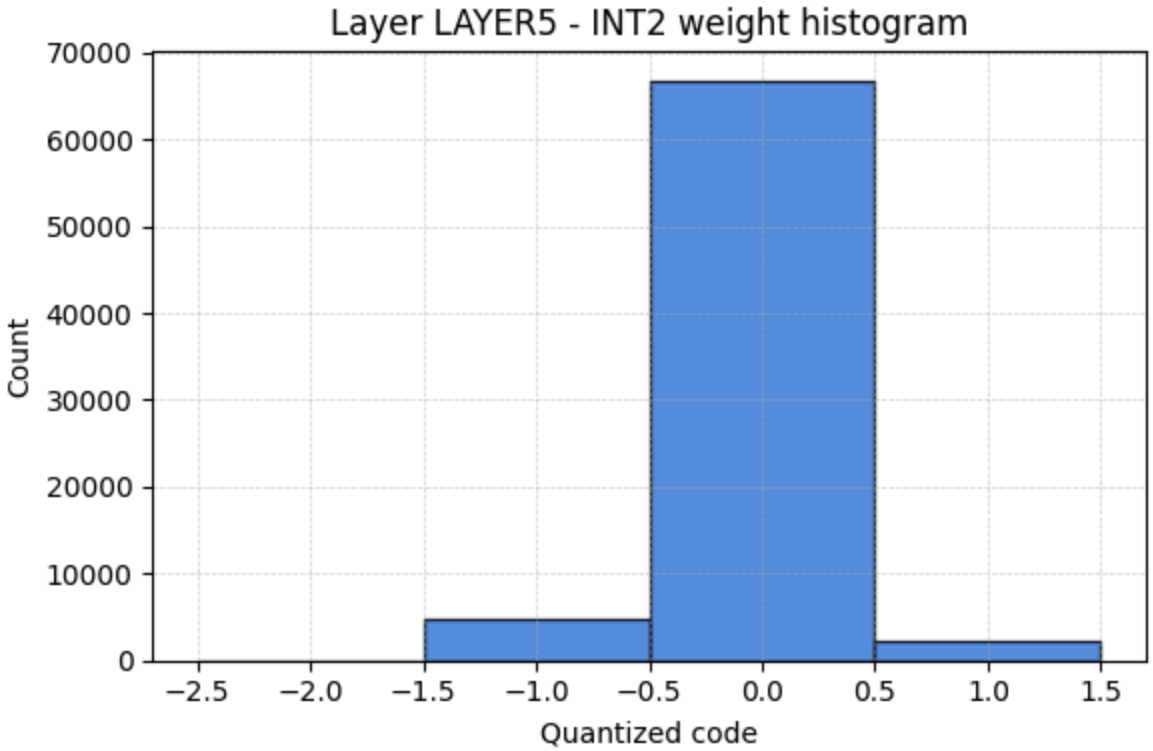

Layer LAYER3 - INT4 weight histogram
Count
Quantized code
0
2000
4000
6000
8000
10000
12000
−8
−6
−4
−2
0
2
4
6
8

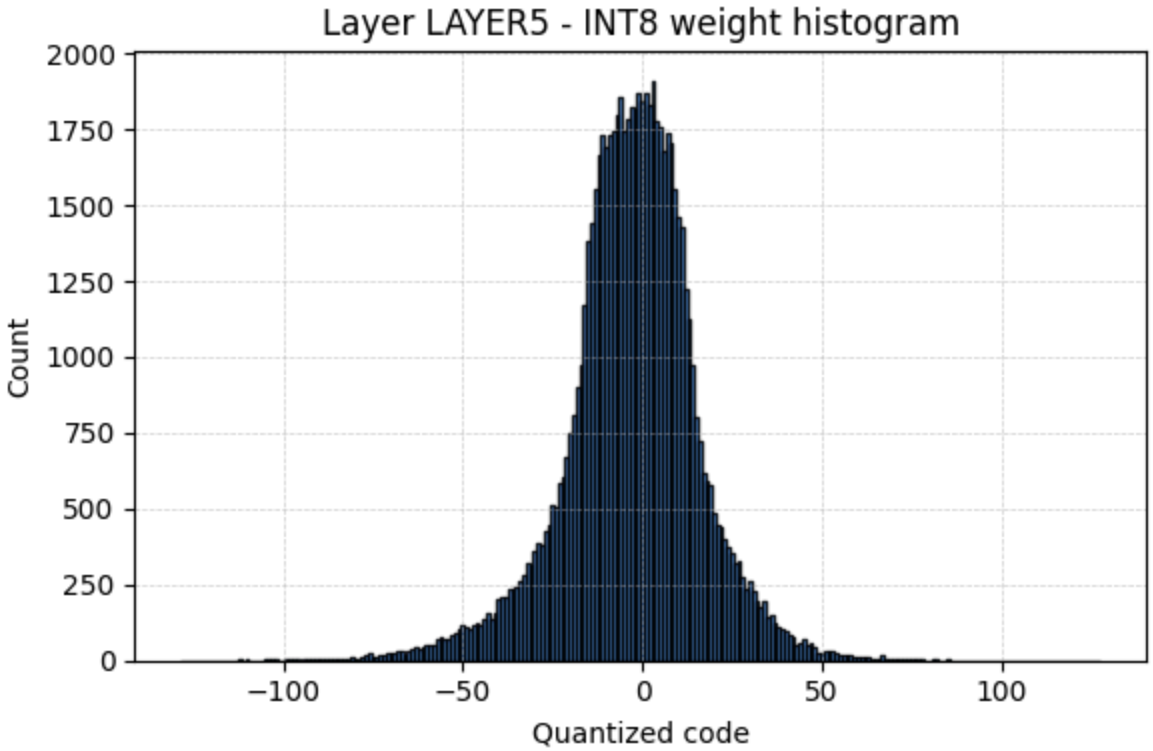

Layer LAYER3 - INT2 weight histogram
Count
Quantized code
0
5000
10000
15000
20000
25000
30000
35000
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5

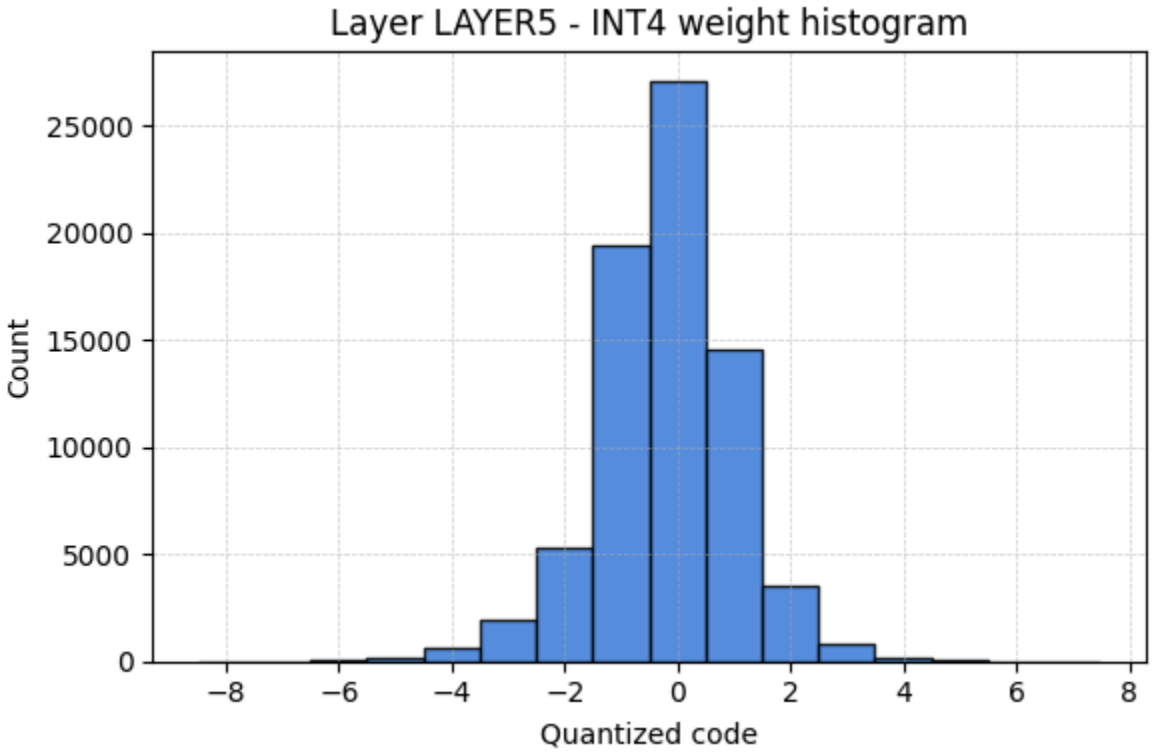

Layer LAYER3 - FP32 weight histogram
Count
Weight value
2500
2000
1500
1000
500
0
−0.4
−0.2
0.0
0.2
0.4


Layer LAYER3 - INT8 weight histogram
Count
Quantized code
0
200
400
600
800
−100
−50
0
50
100


Layer LAYER5 - FP32 weight histogram
Count
Weight value
0
1000
2000
3000
4000
5000
−0.4
−0.2
0.0
0.2
0.4


Layer LAYER5 - INT2 weight histogram
70000
60000
50000
40000
30000
20000
10000
0
Count
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
Quantized code


Layer LAYER5 - INT8 weight histogram
Count
Quantized code
2000
1750
1500
1250
1000
750
500
250
0
−100
−50
0
50
100


Layer LAYER5 - INT4 weight histogram
Count
Quantized code
25000
20000
15000
10000
5000
0
−8
−6
−4
−2
0
2
4
6
8

## Weight Interpretation

Per-channel scaling maintains the dynamic range of every filter. In the first conv layers, we see a wide fp32 spread, but each output channel is centered near zero. Scaling per channel prevents a few energetic filters from overshadowing quieter ones during quantization. The mid and late layers show tighter fp32 ranges, per-channel scaling lets those wider-tailed channels use larger steps without stealing precision.

In INT4, the integer codes fill the full 16 range because a handful of channels still carry large outliers. INT4 survives with a broad and coarse dist. By the time we hit INT2 only four buckets remain, and any heavy tail goes away.

A tiny fraction of weights land at +/-2 while most pile at 0/+-1, signaling that rare outliers already consume the dynamic range and everything else quantizes to a single level. All weights are symmetric at zero.

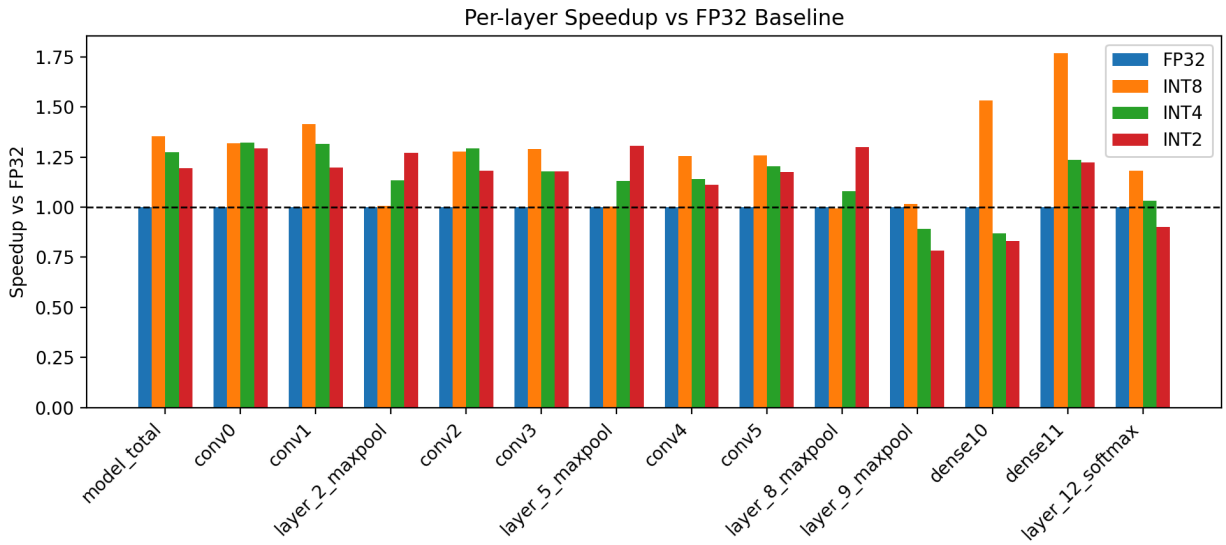
That symmetry arises from using zero as the weight. Filters utilize zero-mean initialization and regularization, which keep positive and negative weights balanced, making symmetric quantizers (ZP=0) work the best.

## Activation Stats

sw > cpre487-587-dnn-framework-main > logs > act_stats.csv

```csv
layer,stage,min,max,clip_99.5
layer_0,pre_relu,-2.49482,1.06899,1.6539
layer_0,post_relu,0,1.06899,0.618924
layer_1,pre_relu,-6.11029,2.27579,4.15993
layer_1,post_relu,0,2.27579,1.21881
layer_10,pre_relu,-16.6554,11.9385,12.6947
layer_10,post_relu,0,11.9385,7.90105
layer_11,pre_relu,-24.0093,7.35558,19.9033
layer_11,post_relu,-24.0093,7.35558,19.9033
layer_2,pre_relu,-4.70016,2.12765,2.6431
layer_2,post_relu,0,2.12765,0.784556
layer_3,pre_relu,-5.04627,2.14972,2.89082
layer_3,post_relu,0,2.14972,0.9382
layer_4,pre_relu,-6.27421,2.84616,4.54372
layer_4,post_relu,0,2.84616,1.82807
layer_5,pre_relu,-12.955,3.55289,8.6553
layer_5,post_relu,0,3.55289,2.40106
```

# Profiling and Speedup Explanation




Each point averages 100 forward passes

End-to-end lat

- FP32 197.1 ms
- INT8 145.5 ms
- INT4 154.7 ms
- INT2 165.2 ms

Conv layers gain the most from quantization - conv1 drops from 108.5 ms (FP32) to 76.6 ms (INT8) and then 82.4 ms (INT4)

Dense on the other hand shows INT8 leading, but INT4/INT2 back off slightly because few non-zero codes still require multiplies after dequantizing - less cache savings

## Timing Stats

```
layer,precision,mean_ms
model_total,fp32,197.964
conv0,fp32,15.999
conv1,fp32,108.934
layer_2_maxpool,fp32,0.217099
conv2,fp32,21.2985
conv3,fp32,34.7479
layer_5_maxpool,fp32,0.0816884
conv4,fp32,6.83213
conv5,fp32,9.13472
layer_8_maxpool,fp32,0.0193879
layer_9_maxpool,fp32,0.00108975
dense10,fp32,0.631691
dense11,fp32,0.0638236
layer_12_softmax,fp32,0.00357406
model_total,int8,147.09
conv0,int8,12.3387
conv1,int8,77.0692
layer_2_maxpool,int8,0.216567
conv2,int8,16.9191
conv3,int8,26.9801
layer_5_maxpool,int8,0.081376
conv4,int8,5.59675
conv5,int8,7.42383
layer_8_maxpool,int8,0.0192699
layer_9_maxpool,int8,0.0010435
dense10,int8,0.406601
dense11,int8,0.0349863
layer_12_softmax,int8,0.0029168
model_total,int4,155.325
conv0,int4,12.1593
conv1,int4,82.811
layer_2_maxpool,int4,0.193087
conv2,int4,16.5407
conv3,int4,29.2255
layer_5_maxpool,int4,0.072315
conv4,int4,5.96772
```

sw > cpre487-587-dnn-framework-main > logs > timing_stats.csv

```
conv4,int4,5.96772
conv5,int4,7.56003
layer_8_maxpool,int4,0.0175234
layer_9_maxpool,int4,0.00107693
dense10,int4,0.72136
dense11,int4,0.0518275
layer_12_softmax,int4,0.00318356
model_total,int2,165.211
conv0,int2,12.3246
conv1,int2,90.8676
layer_2_maxpool,int2,0.167678
conv2,int2,17.9735
conv3,int2,29.2392
layer_5_maxpool,int2,0.0614215
conv4,int2,6.0891
conv5,int2,7.69995
layer_8_maxpool,int2,0.0147837
layer_9_maxpool,int2,0.0011169
dense10,int2,0.717213
dense11,int2,0.051683
layer_12_softmax,int2,0.00315221
```

**Full Dequantization Math**

You can avoid costly FP32 division with M = 2^-n * M0. M0 is represented as a integer (rounded 2^31 * M0). The dequantization is done in the following two steps

1. Multiplying the accumulator by the integer representation of M0 (which is an integer multiplication instead of a FP32)
2. Applying a rounding right bit-shift by n bits (integer shift)

This is functionally the same as long as the bit shift implements correct rounding behavior. This is kind of similar to the Quake inverse square root approximation trick they did where instead of doing costly divisions/square roots they use bit shifting to approximate it. I am not sure if it is really worth doing this to be honest unless we actually are trying to create a really good accelerator.

This is my favorite example of leveraging computer behavior and hardware for performance ever:
https://en.wikipedia.org/wiki/Fast_inverse_square_root

**Validation Set Comparison**

- **Which classes degrade at low bits**

2-bit Accuracy across 1000 images (worse than random guessing)

```
### Calculating Top-1, Top-5, Top-10 Accuracy ###
Using softmax data from: img_data/2bitSoftmax
Top-1 Accuracy: 0.30%
Top-5 Accuracy: 1.40%
Top-10 Accuracy: 3.60%
```

4-bit Accuracy across 1000 images (Better than random guessing!!!)

```
### Calculating Top-1, Top-5, Top-10 Accuracy ###
Using softmax data from: img_data/4bitSoftmax
2025-10-29 23:46:42.023719: W tensorflow/core/kernels/dat
Top-1 Accuracy: 1.30%
Top-5 Accuracy: 4.50%
Top-10 Accuracy: 8.00%
```

8-bit Accuracy: (not too bad if I say so myself [better than the accuracy derived on lab 1 somehow]…)

```
### Calculating Top-1, Top-5, Top-10 Accuracy ###
Using softmax data from: img_data/8bitSoftmax
2025-10-29 23:47:44.354295: W tensorflow/core/kernels
Top-1 Accuracy: 23.70%
Top-5 Accuracy: 49.80%
Top-10 Accuracy: 61.00%
```

# Power and Energy Reports

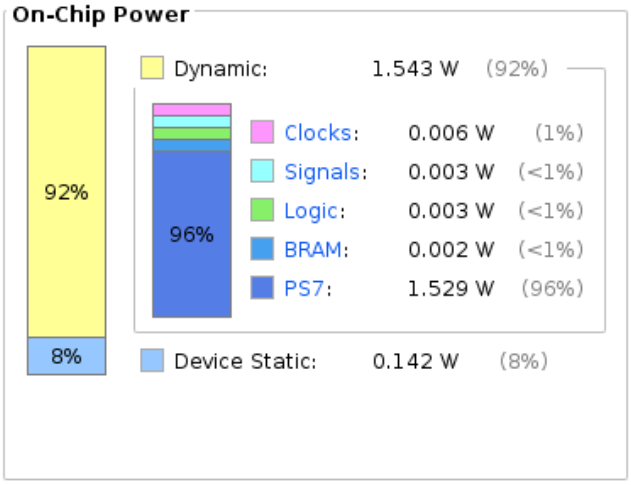
## Power and Energy Report 8-Bit Staged

**Summary**

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.684 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



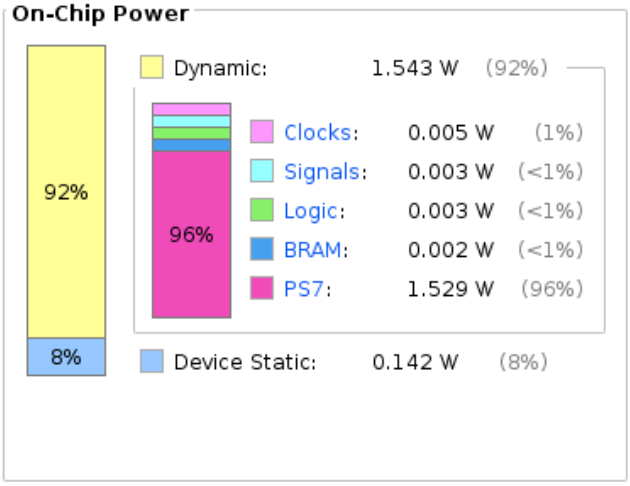

## Power and Energy Report 8-Bit Piped

**Summary**

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.684 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



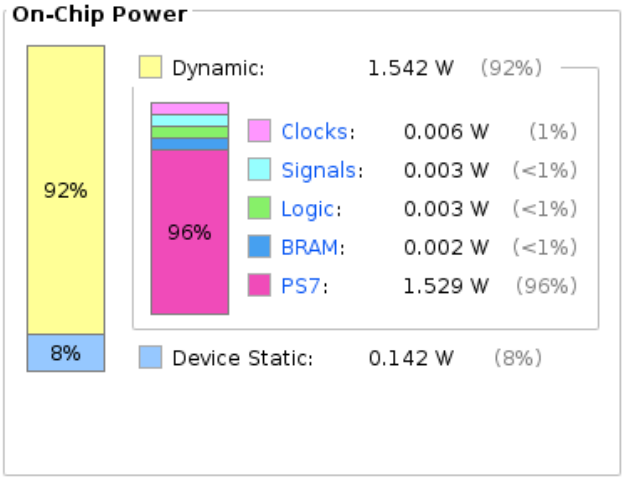
## Power and Energy Report 4-Bit Staged

### Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.684 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



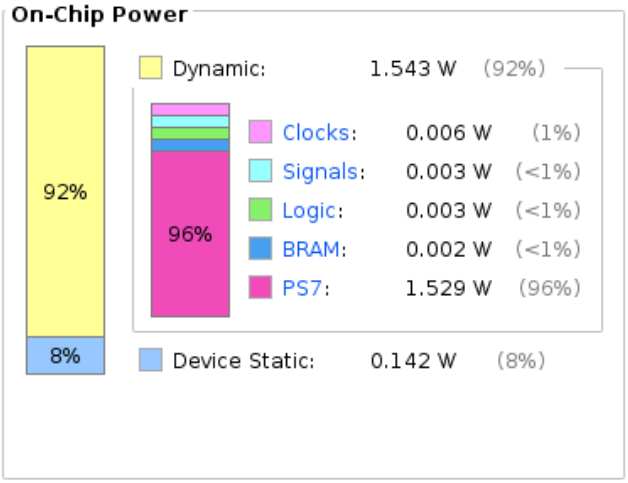

## Power and Energy Report 4-Bit Piped

### Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.685 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



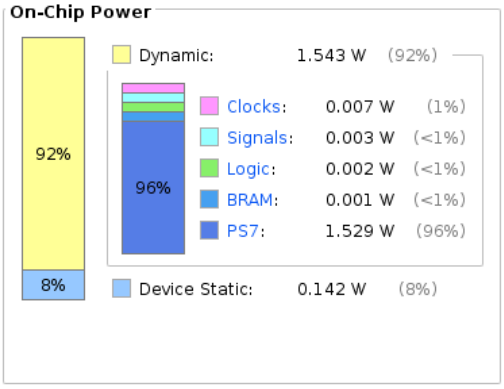
## Power and Energy Report 2-Bit Staged

**Summary**

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.684 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



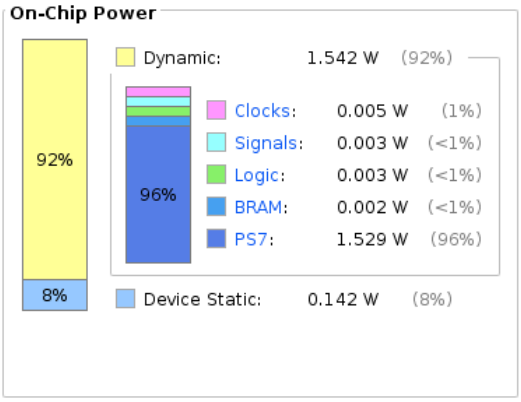

## Power and Energy Report 2-Bit Piped

**Summary**

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.684 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



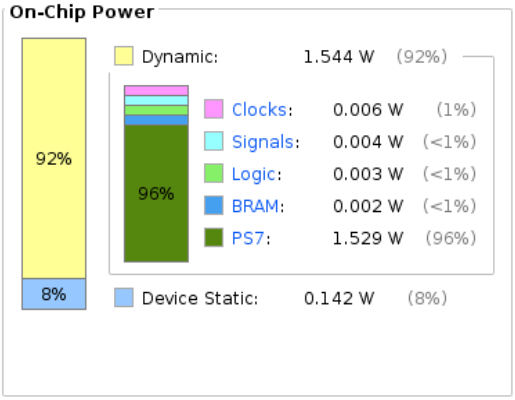

## Power and Energy Report Variable MAC

**Summary**

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

| | |
|---|---|
| **Total On-Chip Power:** | **1.685 W** |
| **Design Power Budget:** | **Not Specified** |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **44.4°C** |
| Thermal Margin: | 40.6°C (3.4 W) |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Medium |

Launch Power Constraint Advisor to find and fix invalid switching activity



# Utilization Reports

## Utilization Report for 8-Bit Staged

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1267 | 1232 | 23 | 480 | 950 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1267 | 1232 | 23 | 480 | 950 | 317 | 3.5 | 0 | 1 |

## Utilization Report for 8-Bit Piped

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1260 | 1247 | 23 | 453 | 943 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1260 | 1247 | 23 | 453 | 943 | 317 | 3.5 | 0 | 1 |

## Utilization Report for 4-Bit Staged

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1225 | 1232 | 24 | 444 | 908 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1225 | 1232 | 24 | 444 | 908 | 317 | 3.5 | 0 | 1 |

## Utilization Report for 4-Bit Piped

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1219 | 1239 | 24 | 466 | 902 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1219 | 1239 | 24 | 466 | 902 | 317 | 3.5 | 0 | 1 |

## Utilization Report for 2-Bit Staged

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1204 | 1232 | 23 | 523 | 887 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1204 | 1232 | 23 | 523 | 887 | 317 | 3.5 | 0 | 1 |

## Utilization Report for 2-Bit Piped

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1201 | 1235 | 23 | 439 | 884 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1201 | 1235 | 23 | 439 | 884 | 317 | 3.5 | 0 | 1 |

## Utilization Report for Variable MAC

| Name | Slice LUTs (53200) | Slice Registers (106400) | F7 Muxes (26600) | Slice (13300) | LUT as Logic (53200) | LUT as Memory (17400) | Block RAM Tile (140) | Bonded IOPADs (130) | BUFGCTRL (32) |
|---|---|---|---|---|---|---|---|---|---|
| staged_mac_bd_wrapper | 1297 | 1250 | 23 | 499 | 980 | 317 | 3.5 | 130 | 1 |
| staged_mac_bd_i (staged_mac_bd) | 1297 | 1250 | 23 | 499 | 980 | 317 | 3.5 | 0 | 1 |

# Inference Reports for 8-Bit, 4-Bit, 2-Bit MACs

## Inference Report for 8-Bit Software

```
[Info]: --- Running Inference Test ---
Opened binary file data/8bit/image_0.bin
Timer Full Inference: elapsed=91.335999ms
Opened binary file data/8bit/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): True 96.1316%  (0.961316)
```

## Inference Report for 8-Bit Staged

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=29167.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 96.1316%  (0.961316)
```

## Inference Report for 8-Bit Piped

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=29167.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 96.1316%  (0.961316)
```

## Inference Report for 4-Bit Software

```
[Info]: --- Running Inference Test ---
Opened binary file data/4bit/image_0.bin
Timer Full Inference: elapsed=89.207001ms
Opened binary file data/4bit/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): False 1.37241%  (0.0137241)
```

## Inference Report for 4-Bit Staged

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/4bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29107.000000ms
xsdb% Opened binary file data/4bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 1.37241%  (0.0137241)
```

## Inference Report for 4-Bit Piped

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/4bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29168.000000ms
xsdb% Opened binary file data/4bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 1.37241%  (0.0137241)
```

**Inference Report for 2-Bit Software**

```
[Info]: --- Running Inference Test ---
Opened binary file data/2bit/image_0.bin
Timer Full Inference: elapsed=93.021004ms
Opened binary file data/2bit/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): False 0.909666%  (0.00909666)
```

**Inference Report for 2-Bit Staged**

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/2bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29099.000000ms
xsdb% Opened binary file data/2bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 0.909666%  (0.00909666)
```

**Inference Report for 2-Bit Piped**

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/2bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29099.000000ms
xsdb% Opened binary file data/2bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 0.909666%  (0.00909666)
```

**Reasons for their differences**

**For starters, our 4 bit software implementation as shown above also has the same low cosine similarity (1.37241%). This means our 4 bit MACs are working and the issue remains in software. I hope this information can provide leniency in grading. :)

Overall our MACs worked and reported the same cosine similarity as our software implementation. This means they are working as expected. There was not much difference between the staged and pipelined macs on each quantization level but this is expected because they were identical in performance even from our Lab 3 MAC designs. From Lab 3, we believe this is due where our pipelines are located.

But as you can see from the results the total time to complete inference went down as the quantization went up (8 bit was the slowest and 2 bit was the fastest). This is expected because the multiplication of two smaller numbers would be less computationally intensive than two larger ones.

# Inference Reports for Variable MACs

### Inference Report for Variable MAC 8-bit

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/8bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29094.000000ms
xsdb% Opened binary file data/8bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 96.1316%  (0.961316)
```

### Inference Report for Variable MAC 4-bit

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/4bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29175.000000ms
xsdb% Opened binary file data/4bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 1.37241%  (0.0137241)
```

### Inference Report for Variable MAC 2-bit

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/2bit/image_0.bin
xsdb% Timer Full Inference: elapsed=29228.000000ms
xsdb% Opened binary file data/2bit/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): False 0.909666%  (0.00909666)
```

**Write-Up on Next Page**

# Variable MAC Write-Up

### MAC Results

From the results, we can conclude that our variable MAC is implemented correctly and works for all three quantization types (8-bit, 4-bit, and 2-bit). Our times for inference are quite interesting and unexpected. When we were creating the variable MAC, our expectations were that it would be slower than 8-bit (**true**) and all three quantization types would have the same inference times (**false**). This turned out to be false and not only are they not the same but the inference time is actually slower for 4-bit and 2-bit. We are still unsure of why this is occurring but it is definitely interesting.

```
if(SD_AXIS_TDATA(16) = '1') then
    multiplied_value <= std_logic_vector(resize(signed(SD_AXIS_TDATA(1 downto 0)) * signed(SD_AXIS_TDATA(3 downto 2)), 16));
elsif(SD_AXIS_ in std_logic_vector(23 downto 0)
    multiplied_value <= std_logic_vector(resize(signed(SD_AXIS_TDATA(3 downto 0)) * signed(SD_AXIS_TDATA(7 downto 4)), 16));
else
    multiplied_value <= std_logic_vector(resize(signed(SD_AXIS_TDATA(7 downto 0)) * signed(SD_AXIS_TDATA(15 downto 8)), 16));
end if;
```

This code block above is the main change from our standard staged MAC. The selection is based on a couple of previously unused bits from our u32 packets. We used bit 16 to indicate we wanted to use 2-bit quantizations. We used bit 17 to indicate we wanted to use 4-bit quantizations. Then if neither were enabled we would use 8-bit quantization.

Overall this design was fairly straightforward, the only issues were with the packet sending (now that we are using some of the unused 16 bits). Also with where I placed this code block initially. Initially I placed this code block in its own process statement, that would update when STDATA updated. This passed in simulation but during integration had issues. I think these issues are caused by when the clock cycle would hit it would be using an outdated multiplied value.

### Quantization Level Selection

We did not actually implement different quantizations for each layer because our 4-bit was not working properly so by doing anything in 4-bit would tank our cosine similarity for the entire inference. Also with our variable MAC implementation, 8-bit is faster than 4-bit and 2-bit so it would be stupid not to choose that for every layer (Unless memory was so important that you don't care about throwing your cosine similarity into the void). But we still want to get at least some points for this section of the lab so we're going to explain our thought process on how we would choose our quantization for each layer.

**Quantization Layer Decisions**
Overall our strategy for the quantization of each layer would be to start with less quantization and increase towards the end of the model. This goal with this would be to get some speed up from the 4-bit while not sacrificing accuracy too early into the inference. Our worry with quantizing too much too early would be that we would lose extra accuracy and this loss of extra accuracy would be propagated through the rest of the inference. We would also never use 2-bit quantization in our layers because it's too inaccurate.

Specifically we would use 8-bit quantization on the first four convolutional layers and then use 4 bit quantization on the last two convolutional layers as well as the dense layers. We believe this strategy would provide a good balance of accuracy and efficiency.

**Theoretical Results**
We would expect our performance, area, power, and energy to be in between 8-bit and 4-bit. Our accuracy would also fall between these two as well. This makes sense because we are using both of them during inference so we get some of the benefits of each.

The main limitation of this design is that accuracy is not going to be greater than 8-bit. Then the energy and area is not going to be better than 4-bit. This makes sense because their maximum benefits occur when we use them for the entire inference. But since we are using them together we get some of the benefits of each but not all of them.